Week 3, Day 5: Kafka Fundamentals, Spring Kafka and Harder Tree Problems

Prepared for Upendra Kumar Sharma | 3.5 to 4 hours | Main topic: Kafka topics, partitions, brokers, producers, consumers, consumer groups, delivery guarantees, Spring Kafka | DSA: Binary tree design and recursion

Today's Schedule

- **0:00 to 0:50** Study Kafka architecture: topics, partitions, brokers and replication (sections 1 and 2).
- **0:50 to 1:40** Study producers and consumers, including consumer groups and offsets (sections 3 and 4).
- **1:40 to 2:10** Study delivery guarantees, retention and when to use Kafka (sections 5 and 6).
- **2:10 to 2:40** Study Spring Kafka configuration and error handling (section 7).
- **2:40 to 3:30** DSA: solve the 2 tree problems in section 9. These are harder, so allow up to 30 minutes each before reading the hint.
- **3:30 to 4:00** Answer the interview questions aloud (section 10) and add 10 flashcards.
- **Optional (90 min):** do the hands-on experiments in section 8. They are the best way to make this topic stick.

1. What Kafka Is

- Kafka is a **distributed, partitioned, replicated commit log**. Producers append records to it and consumers read from it at their own pace.
- **Messages are not deleted when consumed.** They stay for a configured retention period (7 days by default), so many independent consumers can read the same data, and a consumer can **replay** history by resetting its position.
- **Strengths:** very high throughput, durability through replication, horizontal scaling by partitions, ordering within a partition, and replay.
- **Compared with a traditional queue (RabbitMQ, SQS):** a queue usually delivers each message to one consumer and removes it after acknowledgement, and it offers richer per-message routing, priorities and delays. Kafka is a log with per-consumer-group positions, which suits event streams, event-driven microservices, data pipelines, change data capture, log and metrics collection, and stream processing (Kafka Streams, Flink).

2. Core Architecture

Building blocks

- **Broker:** a Kafka server. A **cluster** is several brokers working together.
- **Topic:** a named stream of records, for example orders.created.
- **Partition:** a topic is split into partitions. Each partition is an **ordered, append-only log**, and every record in it has an **offset** (its position number). Partitions are how Kafka scales and parallelizes.
- **Record:** key (optional), value, headers and a timestamp.
- **Segments:** on disk, each partition is a series of segment files, and old segments are deleted or compacted according to the retention policy.

Ordering and keys

- **Ordering is guaranteed only within a partition,** not across a whole topic.
- The producer picks the partition from the record **key** (a hash of the key modulo the number of partitions). **Records with the same key always go to the same partition,** so all events for one order, user or account stay in order.
- Records with **no key** are spread across partitions for balance, and have no ordering relationship.
- **Adding partitions later changes the key-to-partition mapping** for new records, which can break per-key ordering for a while. Plan partition counts ahead.
- **Partition count sets the maximum parallelism** of a consumer group, as section 4 explains.

Replication and fault tolerance

- Each partition has a **replication factor** (typically 3). One replica is the **leader**, and all reads and writes go to it. The others are **followers** that copy the leader.
- The **ISR (in-sync replicas)** are the replicas that are caught up. If the leader fails, a new leader is chosen from the ISR.
- **min.insync.replicas** sets how many replicas must acknowledge a write when the producer uses acks=all. With a replication factor of 3 and min.insync.replicas of 2, the cluster can lose one broker without losing acknowledged data and still accept writes.
- **Cluster metadata and controller:** recent Kafka versions use **KRaft**, with Kafka managing its own metadata. ZooKeeper mode is removed in Kafka 4.0. Mention ZooKeeper only as history.

3. Producers

How a send works: the producer serializes the key and value, chooses a partition, adds the record to a **batch** for that partition, and a background thread sends batches to the partition leader.

Settings to know

- **acks:** how many acknowledgements the producer waits for. **0** means do not wait (fastest, can lose data). **1** means the leader has written it (can lose data if the leader dies before followers copy it). **all** means all in-sync replicas have it (safest).
- **enable.idempotence:** the broker deduplicates retried sends from the same producer, so retries cannot create duplicates or reorder records within a partition. It is on by default in recent versions and needs acks=all.
- **retries and delivery.timeout.ms:** how long the producer keeps trying before reporting failure.
- **Batching and throughput: linger.ms** (wait a few milliseconds to fill larger batches), **batch.size**, and **compression.type** (lz4, zstd, snappy). Bigger batches mean higher throughput and slightly higher latency.
- **max.in.flight.requests.per.connection:** keep it at 5 or below with idempotence to preserve ordering.
- **Transactions (transactional.id):** write to several partitions atomically, and combine consuming and producing into one atomic step (the basis of exactly-once inside Kafka).

A durable configuration to quote in interviews: replication factor 3, min.insync.replicas 2, acks=all, idempotence on, and handling of send failures in a callback.

4. Consumers, Consumer Groups and Offsets

Pull model: consumers **poll** the broker for records, and they control their own pace.

Consumer group (group.id)

- Consumers that share a group id **divide the partitions among themselves**. Each partition is read by **exactly one consumer in the group** at a time. This is how you scale processing.
- If a group has **more consumers than partitions, the extra consumers sit idle**. So the number of partitions is the ceiling on parallelism per group.
- **Different groups are independent.** Each group receives **all** the records, which is how several services consume the same event stream (a publish-subscribe pattern). For example, inventory, billing and notifications are three groups reading the same orders

topic.

Offsets and commits

- Each group tracks its **committed offset** per partition (stored in the internal topic __consumer_offsets). After a restart or a rebalance it resumes from there.
- **Auto commit** periodically commits offsets in the background. It is simple, but can commit records that have not finished processing, which risks **losing** work after a crash.
- **Manual commit** (commitSync or commitAsync, or the acknowledgment modes in Spring) after processing is safer. You then get at-least-once behavior.
- **auto.offset.reset** (earliest, latest, none) only applies when the group has **no committed offset**, such as a brand-new group.

Rebalancing

- When a consumer joins, leaves or crashes, or partitions change, the group **rebalances**: partitions are reassigned. Processing pauses for the affected partitions, so frequent rebalances hurt.
- **Cooperative (incremental) rebalancing** and **static membership (group.instance.id)** reduce the disruption.
- **Heartbeats and timeouts:** a consumer must heartbeat (session.timeout.ms, heartbeat.interval.ms) and must call poll again within **max.poll.interval.ms**. If your processing of one batch takes longer than that, the consumer is removed from the group and the rebalance causes duplicate processing. Fix by processing less per poll (**max.poll.records**), speeding up the handler, or raising the interval.

Consumer lag: the gap between the latest offset in a partition and the group's committed offset. It is the key health metric. **Rising lag means consumers are not keeping up.** Check it with kafka-consumer-groups.sh --describe, and alert on it.

5. Delivery Guarantees

- **At-most-once:** commit the offset **before** processing. A crash loses the message, but it is never processed twice.
- **At-least-once:** process **then** commit. A crash causes reprocessing, so duplicates are possible. **This is the normal choice**, combined with **idempotent consumers** (Day 4).
- **Exactly-once:** Kafka offers it **within Kafka** through idempotent producers and transactions, with consumers using isolation.level=read_committed. As soon as your processing writes to a database or calls an HTTP API, exactly-once for that side effect depends on **your idempotency**, not on Kafka.

- **Ordering rules:** per partition only. To process events of one entity in order, use the entity id as the key. A consumer that retries a failed record in place preserves order but blocks that partition, while sending it to a retry topic keeps things moving but **breaks ordering** for that key.
- **Poison messages:** a record that can never be processed (bad data) must not block the partition forever. Send it to a **dead letter topic (DLT)** after a limited number of retries.

6. Retention, Compaction and When to Use Kafka

- **Retention** by time (retention.ms, default 7 days) or size. Old data is deleted segment by segment.
- **Log compaction (cleanup.policy=compact):** keeps **only the latest record per key**, and a record with a null value (a tombstone) deletes the key. Useful for changelogs, reference data and rebuilding the latest state of each entity.
- **Good fits:** event-driven microservices, outbox relays, CDC pipelines, audit and activity streams, real-time analytics, buffering between systems of different speeds.
- **Poor fits:** a small task queue that needs per-message priorities, delays or complex routing (RabbitMQ or SQS fit better), request-response calls, and very small systems where running a Kafka cluster costs more than it brings. Managed services (Amazon MSK, Confluent Cloud) reduce the operating burden.

7. Spring Kafka

Dependency and configuration: add spring-kafka. Spring Boot auto-configures a KafkaTemplate and listener containers from the spring.kafka properties.

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      acks: all
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer:
org.springframework.kafka.support.serializer.JsonSerializer
      properties:
        enable.idempotence: true
        linger.ms: 5
    consumer:
      group-id: inventory-service
      auto-offset-reset: earliest
      enable-auto-commit: false
      key-deserializer:
```

```
org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer:
org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      properties:
        spring.deserializer.value.delegate.class:
org.springframework.kafka.support.serializer.JsonDeserializer
        spring.json.trusted.packages: com.example.events
    listener:
      ack-mode: record
      concurrency: 3
```

Producing

```
@Service
public class OrderEventPublisher {
    private final KafkaTemplate<String, Object> kafka;
    public OrderEventPublisher(KafkaTemplate<String, Object> kafka) {
this.kafka = kafka; }

    public void publish(OrderCreated event) {
        kafka.send("orders.created", event.orderId(), event)      // key =
order id
             .whenComplete((result, ex) -> {
                 if (ex != null) {
                     // log, alert, and rely on the outbox to retry
                 }
             });
    }
}
```

Consuming

```
@Component
public class OrderEventListener {

    @KafkaListener(topics = "orders.created", groupId = "inventory-service")
    public void onOrderCreated(OrderCreated event) {
        // must be idempotent: the same event can arrive more than once
        inventoryService.reserve(event);
    }
}
```

Error handling and retries

- **DefaultErrorHandler** retries a failed record with a back-off, and then hands it to a **recoverer**. **DeadLetterPublishingRecoverer** publishes it to a topic named after the original with the suffix .DLT, along with headers describing the failure.
- Mark errors that will never succeed (bad payload, validation) as **not retryable**, so they go straight to the DLT.
- **ErrorHandlingDeserializer** wraps your deserializer so that a corrupt record produces a handled error instead of an endless failure loop (a **poison pill** problem).
- **@RetryableTopic** gives non-blocking retries through separate retry topics, at the cost of ordering.
- **Concurrency** sets how many consumer threads run in one instance, and it should not exceed the number of partitions.
- **Acknowledgment modes:** RECORD (commit after each record), BATCH (after each poll batch), MANUAL (you call acknowledge).

```
@Bean
DefaultErrorHandler errorHandler(KafkaTemplate<Object, Object> template) {
    var recoverer = new DeadLetterPublishingRecoverer(template);
    var handler = new DefaultErrorHandler(recoverer, new FixedBackOff(1000L,
3L));
    handler.addNotRetryableExceptions(IllegalArgumentException.class);
    return handler;
}
```

Testing: use **Testcontainers** with a real Kafka container, or the embedded Kafka from spring-kafka-test, and assert on behavior, including duplicate delivery and the DLT path.

8. Optional Hands-On (90 Minutes)

Start Kafka with Docker Compose (single node, KRaft mode). Check the official image page for the latest version tag.

```
services:
  kafka:
    image: apache/kafka:3.8.0
    ports:
      - 9092:9092
```

Command-line tools (run them inside the container, in /opt/kafka/bin)

```
docker compose exec kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server
localhost:9092 --create --topic orders.created --partitions 3
docker compose exec kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server
localhost:9092 --describe --topic orders.created
docker compose exec kafka /opt/kafka/bin/kafka-consumer-groups.sh --
bootstrap-server localhost:9092 --describe --group inventory-service
```

Experiments, and what to observe

1. Create a topic with 3 partitions. Run **one** consumer of a group and note that it owns all three partitions.
2. Start a **second and third** instance of the same group. Watch the partitions spread. Start a **fourth** and confirm it stays idle.
3. Send messages with the **same key** repeatedly and verify they all land in one partition, in order. Send with **different keys** and watch the spread.
4. **Kill** a consumer and watch the rebalance and which instance takes over.
5. Stop all consumers, produce 50 messages, restart one, and confirm it **resumes from the committed offset**. Check the lag before and after.
6. Make the handler throw **after processing but before the commit**, kill the app, and see the **duplicate** on restart. This is at-least-once, and why idempotency matters.
7. Send a **malformed** message and watch it reach the .DLT topic after the retries.
8. Replace the logging stub in your Day 4 **outbox relay** with the KafkaTemplate, using the aggregate id as the key. Your order service now publishes reliably.

Write down what you saw. Concrete observations make your answers about rebalancing and ordering convincing.

9. DSA: Binary Tree Design and Recursion (2 Problems)

These use the TreeNode class from Day 3, and they are harder than the earlier tree problems.

Problem 1: Serialize and Deserialize Binary Tree

- **Idea:** write the tree in **preorder**, using a marker such as # for null children. The preorder sequence with null markers identifies the tree uniquely. To rebuild, read tokens in the same order: take a token, create the node, then recursively build its left and right subtrees from the following tokens.
- **Complexity:** O(n) time and O(n) space for both directions.
- **Edge cases:** an empty tree (a single null marker), negative values, a skewed tree (deep recursion, so mention an iterative or level-order approach for huge trees).

- **Alternative:** a level-order (BFS) encoding, which is the format LeetCode shows for trees.
- **Why it is asked:** it checks recursion over structure and the idea of a self-delimiting format, which also appears in real serialization work.

```
public String serialize(TreeNode root) {
    StringBuilder sb = new StringBuilder();
    write(root, sb);
    return sb.toString();
}

private void write(TreeNode node, StringBuilder sb) {
    if (node == null) {
        sb.append("#,");
        return;
    }
    sb.append(node.val).append(',');
    write(node.left, sb);
    write(node.right, sb);
}

public TreeNode deserialize(String data) {
    Deque<String> tokens = new ArrayDeque<>(Arrays.asList(data.split(",")));
    return read(tokens);
}

private TreeNode read(Deque<String> tokens) {
    String token = tokens.poll();
    if (token.equals("#")) return null;
    TreeNode node = new TreeNode(Integer.parseInt(token));
    node.left = read(tokens);
    node.right = read(tokens);
    return node;
}
```

Problem 2: Binary Tree Maximum Path Sum

- **Idea:** a path can start and end at **any** nodes and does not have to pass through the root. Use a **postorder** recursion where each call returns the **best downward path starting at that node** (the node plus its better child branch, ignoring any branch whose sum is negative). While doing that, **update a global best** with the best path that **turns** at this node, which is node value plus the left gain plus the right gain.
- **Pattern to remember:** return one thing to the parent, record a different thing globally. The same pattern solves Diameter of Binary Tree.

- **Complexity:** O(n) time, O(h) space.
- **Edge cases:** all values negative (the answer is the largest single node, so start the best at the minimum integer and update with the node value), a single node.
- **Note:** the field below makes the class non-thread-safe, which is fine for an interview, but say so.

```
private int best = Integer.MIN_VALUE;

public int maxPathSum(TreeNode root) {
    gain(root);
    return best;
}

private int gain(TreeNode node) {
    if (node == null) return 0;
    int left = Math.max(0, gain(node.left));      // ignore negative branches
    int right = Math.max(0, gain(node.right));
    best = Math.max(best, node.val + left + right);   // path that turns at
this node
    return node.val + Math.max(left, right);          // path that continues
upward
}
```

10. Interview Questions to Answer Aloud

1. What is Kafka, and how does it differ from a traditional message queue such as RabbitMQ?
2. Explain topics, partitions, offsets and brokers. Where is ordering guaranteed?
3. How does a producer choose a partition? What happens if you add partitions later?
4. Explain replication, leader and followers, ISR and min.insync.replicas. What configuration prevents data loss when one broker fails?
5. What do acks 0, 1 and all mean? What does the idempotent producer add?
6. What is a consumer group? What happens when you have more consumers than partitions?
7. How do offsets and commits work? Compare auto commit with manual commit.
8. What causes a rebalance, and how do you reduce its impact? What happens when processing exceeds max.poll.interval.ms?
9. At-most-once, at-least-once and exactly-once: what does Kafka really guarantee, and what is left to you?

10. How do you keep events for one entity in order, and what happens to ordering when you use retry topics?
11. How do you handle a poison message? Explain the dead letter topic.
12. What is consumer lag, and what would you do if it keeps growing?
13. What is log compaction, and when is it useful?
14. How would you publish events reliably from a service that also writes to a database?

11. End-of-Day Checklist

- [ ] I can explain topics, partitions, offsets, brokers, replication and ISR.
- [ ] I can explain partition keys and exactly where ordering is guaranteed.
- [ ] I can state a durable producer configuration and explain acks and idempotence.
- [ ] I can explain consumer groups, offsets, rebalancing, lag and the three delivery semantics.
- [ ] I can configure Spring Kafka with a listener, a template and a DLT error handler.
- [ ] I solved both tree problems and can explain the return-one-thing, record-another pattern.
- [ ] (Optional) I ran the Kafka experiments and recorded what I observed.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Week 3, Day 6, weekend): start the mini project: three services, Docker Compose, JWT at the gateway, and the skeleton from Days 1 to 5.